



1. (20 نمره)

الف- (10 نمره) تابعی بنویسید که ابعاد یک ماتریس مربعی N و خود ماتریس $matrix$ را دریافت کند. این تابع باید مجموع عناصر موجود در مثلث بالایی (شامل قطر اصلی و عناصر بالای آن) را محاسبه کرده و برگرداند. (نکته: فقط مجاز به استفاده از اشاره گر هستید و نباید از اندیس‌گذاری آرایه $[]$ استفاده کنید)

```
int sumUpperTriangle(int N, int matrix[N][N]) {  
  
}
```

```
int sumUpperTriangle(int N, int matrix[N][N]) {  
    int sum = 0;  
    for (int i = 0; i < N; i++) {  
        for (int j = i; j < N; j++) { // Upper triangle means j>=i  
            sum += (*(matrix + i) + j);  
        }  
    }  
  
    return sum;  
}
```

ب- (10 نمره) تابعی بنویسید که ابعاد یک ماتریس مربعی N و خود ماتریس $matrix$ را دریافت کند. این تابع باید بررسی کند آیا ماتریس ورودی یک ماتریس همانی است یا خیر. اگر بود 1 و در غیر این صورت 0 برگرداند. (فقط مجاز به استفاده از اشاره گر هستید)
(راهنمایی: ماتریس همانی ماتریسی است که تمام عناصر روی قطر اصلی آن 1 و بقیه عناصر 0 باشند)

```
int isIdentityMatrix(int N, int matrix[N][N]) {  
  
}
```

```

int isIdentityMatrix(int N, int matrix[N][N]) {
    int i, j;

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {

            int value = (*(matrix + i) + j);
            if (i == j) {
                if (value != 1) { // elements on main diagonal must be 1
                    return 0;
                }
            }
            else {
                if (value != 0) { // other elements must be 0
                    return 0;
                }
            }
        }
    }
    return 1;
}

```

2. (20 نمره) تابعی به نام deleteElement بنویسید. این تابع سه پارامتر را به صورت اشاره‌گر

دریافت می‌کند تا بتواند عملیات زیر را بر روی آرایه ورودی انجام دهد:

جستجو: عنصر با مقدار *target را در آرایه‌ای که از آدرس addr شروع می‌شود، پیدا کند.

حذف و شیفت: اگر عنصر پیدا شد، اولین رخداد آن را حذف کند؛ یعنی تمام عناصر موجود در

خانه‌های بعدی باید یک خانه به سمت چپ جابجا (شیفت) شوند.

به‌روزرسانی سایز: در صورت موفقیت‌آمیز بودن حذف، مقدار متغیر *size (که نشان‌دهنده تعداد

عناصر موجود در آرایه است) باید یک واحد کاهش یابد.

```

void deleteElement(int *addr, int *target, int *size) {

```

```

}

```

```

void delete_element(int *addr, int *target, int *size) {
    int i;
    int index = -1;
    for (i = 0; i < *size; i++) {
        if (*(addr+i) == *target) {
            index = i; // found the target at index i
            break;
        }
    }
    // if the target is found, shift others by one to left
    if (index != -1) {
        for (i = index; i < *size - 1; i++) {
            *(addr+i) = *(addr+i+1);
        }
        (*size)--; // decrease size of array
    }
}

```

3. (10 نمره) یک مزیت از pass by value و یک مزیت از pass by reference را بیان کنید.

یک مثال از شرایطی بزنید که pass by reference نسبت به pass by value بهتر است (نیاز به نوشتن کد نیست).

در pass by value، از آنجا که تابع با یک کپی از مقدار اصلی کار می‌کند، هر تغییری که در داخل تابع روی آن مقدار ایجاد شود، هیچ تأثیری بر روی متغیر اصلی خارج از تابع نخواهد داشت. این امر تغییرات ناخواسته را کاهش می‌دهد.

در pass by reference، به جای کپی کردن کل داده، تنها آدرس حافظه (اشاره‌گر) آن ارسال می‌شود. این کار باعث می‌شود که در زمان انتقال داده‌های بزرگ، زمان و حافظه‌ی کمتری صرف کپی کردن شود. فرض کنید یک ساختار داده بزرگ (مثلاً حاوی ۱۰۰,۰۰۰ عدد) دارید و می‌خواهید تابعی بنویسید تا تغییراتی روی این داده‌ها اعمال کند.

- اگر از **Pass by Value** استفاده کنید، سیستم باید کل ۱۰۰,۰۰۰ عنصر را کپی کند که زمان و حافظه‌ی زیادی را هدر می‌دهد.

- اگر از **Pass by Reference** استفاده کنید، فقط آدرس ساختار داده ارسال می‌شود و تابع مستقیماً روی داده‌های اصلی کار می‌کند.

بنابراین، برای پردازش آرایه‌ها و ساختارهای داده حجیم، استفاده از pass by reference از نظر کارایی به مراتب بهتر است. (آرایه‌ها همواره به صورت pass by reference منتقل می‌شوند)

4. امتیازی** (15 نمره) فرض کنید `arr` یک آرایه با اعداد صحیح به طول `n` است که به دو زیر بخش تقسیم شده است به طوری که هر دو زیر بخش مرتب شده هستند. فرض کنید زیر بخش اول از اندیس 0 تا اندیس `r` است و زیر بخش دوم از اندیس `r + 1` تا اندیس `n - 1` است. تابع زیر را به گونه ای تکمیل کنید تا این آرایه `arr` به همراه دو پارامتر دیگر `r` و `n` را بگیرد و این آرایه را به طور کامل مرتب کند. همچنین برای حل این سوال از اشاره گر ها استفاده کنید.

```
void mergeSubarrays(int *arr, int r, int n) {  
  
}
```

راهنمایی: از اندیس `r + 1` شروع کنید و عناصر را تک تک به زیر بخش اول اضافه کنید، طوری که هر دو زیر بخش، به صورت مرتب شده باقی بمانند.

```
void mergeSubarrays(int *arr, int r, int n) {  
  
    int i, j;  
    for (i = r + 1; i < n; i++) {  
  
        int key = *(arr + i);  
        j = i - 1;  
  
        while (j >= 0 && *(arr + j) > key) {  
            *(arr + (j + 1)) = *(arr + j);  
            j--;  
        }  
  
        *(arr + (j + 1)) = key;  
    }  
}
```

5. امتیازی** (25 نمره) ابتدا توضیح دهید که فرق بین `int (*arr1)[5]` و `int *arr2[5]` چیست، سپس با توضیح خط به خط خطوط 1 تا 12 (مشخص شده با کامنت) ، خروجی برنامه را نمایش دهید.

راهنمایی: وقتی میگوییم `a` یک آرایه به طول `n` است، یعنی `a` به بخشی از حافظه اشاره می‌کند که از `n` خانه متوالی تشکیل شده و پشت سر هم قرار گرفته اند. برای دسترسی به آدرس عناصر، از نام این متغیر به همراه عملگر های جمع و تفریق استفاده میشود. مثلاً:

`a` یعنی آدرس اولین خانه

`a + 3` یعنی آدرس چهارمین خانه

اگر `b` یک اشاره گر به پنجمین خانه حافظه آرایه `a` باشد، `b - 2` یعنی آدرس سومین خانه آرایه `a` و برای دسترسی به مقدار عناصر، میتوانیم از `[ ]` یا `*` استفاده کنیم. مثلاً:

`a[2]` یعنی مقدار سومین خانه

`*(a + 1)` یعنی مقدار دومین خانه

مثال از آرایه تک بعدی و دو بعدی:

$$a + i \equiv \&a[i] , *(a + i) + j \equiv a[i] + j \equiv \&a[i][j]$$

$$*(a + i) \equiv a[i] , (*(a + i) + j) \equiv *(a[i] + j) \equiv a[i][j]$$

```
#include <stdio.h>

void function(int (*arr1)[5], int *arr2[5]) {
    int *p = (*arr1) + 1;           // 3
    int *q = arr2[2] - 1;           // 4

    (*arr1)[2] = arr2[0][1] + *p;    // 5
    arr2[1] = (*arr1) + 3;           // 6

    p = *arr1 + 3;                   // 7
    *arr2[1] += q[1];                // 8

    *(arr2 + 3) = arr2[1] - 3;       // 9
    q = arr2[4];                     // 10

    *(arr2[3] + 2) = arr2[1][0] + (*arr1)[0] + q[1]; // 11
    *p = arr2[3][2] - (*(arr2 - 1)); // 12
}
```

```

int main() {
    int x[5] = {3, 0, 1, 7, 6};
    int y[5] = {9, 4, 8, 5, 2};

    int (*arr1)[5] = &x; // 1
    int *arr2[5] = {y + 3, &x[2], &y[4], x + 1, y}; // 2

    function(arr1, arr2);

    for(int i = 0; i < 5; i++){
        printf("%d\t", x[i]);
    }
    printf("\n");

    for(int i = 0; i < 5; i++){
        printf("%d\t", *(y + i));
    }
    printf("\n");

    for(int i = 0; i < 5; i++){
        printf("%d\t", *(arr2[i]));
    }
    printf("\n");

    return 0;
}

```

پاسخ : `int (*arr1)[5]` یک اشاره گر به یک آرایه 5 تایی است ولی `int *arr2[5]` تعریف یک آرایه 5 تایی است که عناصر آن، اشاره گر به یک عدد صحیح هستند.

برای ارتقای کیفیت درک هر خط، هر سه آرایه `x` و `y` و `arr2` را پس از پایان هر خط نمایش میدهیم.

همچنین اشاره گر `p` و `q` را به ترتیب با رنگ های **قرمز** و **سبز** نمایش میدهیم.

خط 1) `arr1` یک اشاره گر به آرایه است. در این قسمت `arr1` را با آدرس آرایه `x` مقداردهی اولیه کردیم.

خط 2) `arr2` یک آرایه است که مقدار هر عنصر از آن، یک آدرس به یک عدد صحیح است. با توجه به مقداردهی صورت گرفته، محتوای آن شامل 5 عنصر است که هر کدام به یک عدد صحیح اشاره میکنند.

`x = {3, 0, 1, 7, 6}`

`y = {9, 4, 8, 5, 2}`

`arr2 = {y + 3, x + 2, y + 4, x + 1, y}`

خط 3) p یک اشاره گر به یک عدد صحیح است. $(*arr1)$ برابر با مقدار $arr1$ است که این مقدار برابر با x است، همچنین $(*arr1) + 1$ برابر با $x + 1$ است (آدرس خانه دوم آرایه x). سپس p را با این آدرس مقاردهی میکنیم.

$x = \{3, 0, 1, 7, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 2, y + 4, x + 1, y\}$

خط 4) q یک اشاره گر به یک عدد صحیح است. $arr2[2]$ برابر با مقدار خانه سوم آرایه $arr2$ است که این مقدار برابر با $y + 4$ است (آدرس خانه پنجم آرایه y). همچنین $arr2[2] - 1$ برابر با $y + 3$ است (آدرس خانه چهارم آرایه y). سپس q را با این آدرس مقاردهی میکنیم.

$x = \{3, 0, 1, 7, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 2, y + 4, x + 1, y\}$

خط 5) $(*arr1)$ معادل مقدار $arr1$ است که این مقدار برابر با x است پس $(*arr1)[2]$ همان $x[2]$ و برابر با مقدار خانه سوم آرایه x است. $arr2[0]$ برابر با مقدار خانه اول آرایه $arr2$ است که برابر با $y + 3$ است. پس $arr2[0][1]$ برابر با $(y + 3)[1]$ است که این نیز برابر با $y[4]$ است (مقدار خانه پنجم آرایه y). بدیهی است که $*p$ نیز برابر با مقدار p است. در نهایت مجموع دو عبارت سمت راست (2) را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 2, 7, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 2, y + 4, x + 1, y\}$

خط 6) $arr2[1]$ برابر با مقدار خانه دوم آرایه $arr2$ است که برابر با $x + 2$ است (آدرس خانه سوم آرایه x) است. $(*arr1)$ نیز برابر با مقدار $arr1$ است که این مقدار برابر با x است. در نهایت مجموع 2 عبارت سمت راست $(x + 3)$ را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 2, 7, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x + 1, y\}$

خط 7) p یک اشاره گر به یک عدد صحیح است. $*arr1$ برابر با مقدار $arr1$ است که برابر با x است. در نهایت مجموع 2 عبارت سمت راست $(x + 3)$ را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 2, 7, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x + 1, y\}$

خط 8) میدانیم که اولویت $[]$ از $*$ بیشتر است. $arr2[1]$ برابر با مقدار خانه دوم آرایه $arr2$ است که برابر با $x+3$ است. $*arr2[1]$ برابر با مقدار $x + 3$ یا همان $x[3]$ است (مقدار خانه چهارم آرایه x). q برابر است با $y+3$ است؛ پس $q[1]$ برابر با مقدار $y + 4$ است (مقدار خانه پنجم آرایه y). در نهایت نتیجه عبارت سمت راست (9) را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 2, 9, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x + 1, y\}$

خط 9) $(arr2 + 3)$ برابر با مقدار خانه چهارم آرایه $arr2$ است که برابر با $x + 1$ است. $arr2[1]$ نیز برابر با مقدار خانه دوم آرایه $arr2$ است که برابر با $x + 3$ است. در نهایت مجموع 2 عبارت سمت راست (x) را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 2, 9, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x, y\}$

خط 10) q یک اشاره گر به یک عدد صحیح است. $arr2[4]$ برابر با مقدار خانه پنجم آرایه $arr2$ است که برابر با y است (آدرس خانه اول آرایه y). در نهایت نتیجه عبارت سمت راست (y) را در عبارت چپ میریزیم.

$x = \{3, 0, 2, 9, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x, y\}$

خط 11) $arr2[3]$ برابر با مقدار خانه چهارم آرایه $arr2$ است که برابر با x است، همچنین $arr2[3] + 2$ برابر با $x + 2$ است (آدرس خانه سوم آرایه x)، در نهایت $(arr2[3] + 2) *$ برابر با مقدار خانه سوم آرایه x است. $arr2[1]$ برابر با مقدار خانه دوم آرایه $arr2$ است که برابر با $x + 3$ است. پس $arr2[1][0]$ برابر با $(x + 3)[0]$ است که این نیز برابر با $x[3]$ است (مقدار خانه چهارم آرایه x). $(*arr1)$ برابر با مقدار $arr1$ است که برابر با x است پس $[0](*arr1)$ برابر با $x[0]$ است (مقدار خانه اول آرایه x). q برابر با y است پس $q[1]$ برابر با مقدار $y + 1$ است (مقدار خانه دوم آرایه y). در نهایت مجموع 3 عبارت سمت راست (16) را در عبارت سمت چپ میریزیم.

$x = \{3, 0, 16, 9, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x, y\}$

خط 12) $*p$ برابر با مقدار p است. $arr2[3]$ برابر با مقدار خانه چهارم آرایه $arr2$ است که برابر با x است. پس $arr2[3][2]$ برابر با $x[2]$ است (مقدار خانه سوم آرایه x). $*arr2$ برابر با مقدار خانه اول آرایه $arr2$ است که برابر با $y + 3$ است، پس $arr2 - 1$ برابر با $y + 2$ است (آدرس خانه سوم آرایه y). پس $(*arr2 - 1)$ برابر با مقدار خانه سوم آرایه y است. در نهایت مجموع 2 عبارت سمت راست (8) را به عبارت سمت چپ اضافه میکنیم.

$x = \{3, 0, 16, 8, 6\}$

$y = \{9, 4, 8, 5, 2\}$

$arr2 = \{y + 3, x + 3, y + 4, x, y\}$

خروجی نهایی:

3 0 16 8 6

9 4 8 5 2

5 8 2 3 9